

# Self-Hosted / VPS Deployment Guide — Nucleus AI

---

Deploy Nucleus AI on any VPS provider (Hetzner, OVH, Linode, Vultr, etc.) or on-premises.

---

## Prerequisites

---

- A Linux server (Ubuntu 22.04+ recommended) with:
  - Minimum 4 GB RAM, 2 vCPUs, 40 GB disk
  - Recommended: 8 GB RAM, 4 vCPUs, 80 GB SSD
  - Root or sudo access
  - A domain name with DNS pointing to your server IP
  - Ports 80 and 443 open
- 

## Quick Start

---

Use the automated deployment script:

```
curl -fsSL https://raw.githubusercontent.com/your-org/nucleus-ai/main/scripts/deployment/deploy.sh | bash
```

Or follow the manual steps below.

---

## Step 1: Server Preparation

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install required packages
sudo apt install -y curl git ufw fail2ban

# Configure firewall
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow http
sudo ufw allow https
sudo ufw enable

# Install Docker
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version
docker compose version
```

## Step 2: Clone and Configure

```
# Clone repository
git clone https://github.com/your-org/nucleus-ai.git /opt/nucleus-ai
cd /opt/nucleus-ai

# Create production environment file
cp .env.production.example .env.production
```

Edit `.env.production` with your values:

```
nano .env.production
```

### Required changes:

- `DOMAIN` — your domain name
- `POSTGRES_PASSWORD` — generate with `openssl rand -hex 32`
- `SECRET_KEY` — generate with `openssl rand -hex 64`
- `NEXTAUTH_SECRET` — generate with `openssl rand -base64 32`
- `OPENAI_API_KEY` — your OpenAI API key
- `NEXT_PUBLIC_API_URL` — `https://yourdomain.com/api`
- `NEXTAUTH_URL` — `https://yourdomain.com`

## Step 3: SSL Certificates with Let's Encrypt

```
# Install Certbot
sudo apt install certbot -y

# Obtain certificate (make sure ports 80/443 are not in use)
sudo certbot certonly --standalone -d yourdomain.com -d www.yourdomain.com

# Copy certificates
sudo cp /etc/letsencrypt/live/yourdomain.com/fullchain.pem /opt/nucleus-ai/nginx/ssl/
sudo cp /etc/letsencrypt/live/yourdomain.com/privkey.pem /opt/nucleus-ai/nginx/ssl/
sudo chmod 644 /opt/nucleus-ai/nginx/ssl/*.pem
```

### Auto-Renewal

```
# Create renewal hook
sudo tee /etc/letsencrypt/renewal-hooks/deploy/nucleus-ai.sh << 'EOF'
#!/bin/bash
cp /etc/letsencrypt/live/yourdomain.com/fullchain.pem /opt/nucleus-ai/nginx/ssl/
cp /etc/letsencrypt/live/yourdomain.com/privkey.pem /opt/nucleus-ai/nginx/ssl/
cd /opt/nucleus-ai && docker compose -f docker-compose.prod.yml restart nginx
EOF
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/nucleus-ai.sh

# Test renewal
sudo certbot renew --dry-run
```

## Step 4: Update Nginx Configuration

```
# Update server_name in nginx config
sed -i 's/yourdomain.com/YOUR_ACTUAL_DOMAIN/g' nginx/conf.d/default.conf
```

## Step 5: Deploy

```
# Build and start all services
docker compose -f docker-compose.prod.yml --env-file .env.production up -d --build

# Check status
docker compose -f docker-compose.prod.yml ps

# View logs
docker compose -f docker-compose.prod.yml logs -f
```

## Step 6: Run Database Migrations

```
# Run Alembic migrations for backend
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head

# Run Prisma migrations for frontend
docker compose -f docker-compose.prod.yml exec frontend npx prisma migrate deploy
```

## Step 7: Verify Deployment

---

```
# Test health endpoints
curl -s https://yourdomain.com/health | jq .
curl -s https://yourdomain.com/api/v1/health | jq .

# Check all containers are healthy
docker compose -f docker-compose.prod.yml ps
```

---

## Server Hardening

### SSH Hardening

```
# Disable password authentication
sudo sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/
sshd_config
sudo sed -i 's/PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/
sshd_config
sudo systemctl restart sshd
```

### Fail2Ban Configuration

```
sudo tee /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 5

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log

[nginx-http-auth]
enabled = true
logpath = /var/log/nginx/error.log
EOF

sudo systemctl enable fail2ban
sudo systemctl restart fail2ban
```

### Automatic Updates

```
sudo apt install unattended-upgrades -y
sudo dpkg-reconfigure -plow unattended-upgrades
```

---

## Backup Strategy

---

### Automated Database Backups

```
# Create backup script
sudo tee /opt/nucleus-ai/scripts/backup.sh << 'SCRIPT'
#!/bin/bash
set -euo pipefail
BACKUP_DIR=/opt/backups/nucleus-ai
mkdir -p $BACKUP_DIR
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# PostgreSQL backup
docker compose -f /opt/nucleus-ai/docker-compose.prod.yml exec -T postgres \
  pg_dump -U nucleus nucleus | gzip > ${BACKUP_DIR}/db_${TIMESTAMP}.sql.gz

# Keep last 7 days
find $BACKUP_DIR -name "db_*.sql.gz" -mtime +7 -delete

echo "Backup completed: ${BACKUP_DIR}/db_${TIMESTAMP}.sql.gz"
SCRIPT

sudo chmod +x /opt/nucleus-ai/scripts/backup.sh

# Schedule daily backups at 2 AM
echo "0 2 * * * /opt/nucleus-ai/scripts/backup.sh" | sudo crontab -
```

---

## Updating

---

```
cd /opt/nucleus-ai

# Pull latest code
git pull origin main

# Rebuild and restart
docker compose -f docker-compose.prod.yml --env-file .env.production up -d --build

# Run migrations
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head
```

---

## Troubleshooting

| Issue                       | Solution                                                                      |
|-----------------------------|-------------------------------------------------------------------------------|
| Container won't start       | <code>docker compose logs &lt;service&gt;</code>                              |
| Database connection refused | Check PostgreSQL health: <code>docker compose exec postgres pg_isready</code> |
| 502 Bad Gateway             | Backend may still be starting — check health endpoint                         |
| SSL certificate error       | Verify certificate files exist in <code>nginx/ssl/</code>                     |
| Out of memory               | Increase server RAM or reduce worker count                                    |
| Disk full                   | Clean Docker: <code>docker system prune -a</code>                             |